Python OOP Practice Tasks

HackerRank Style • Topic: Classes, Objects & State Management

Task 1: Shopping Cart

Easy • OOP Basic

PROBLEM STATEMENT

Design a class named `ShoppingCart` to manage items and calculate totals in a shopping cart.

Your class needs to support:

- Initializing with a customer's name.
- An `add_item(item_name, price)` method that takes an item name (string) and its price (float).
- A `get_total()` method that returns the total cost of all items currently in the cart.

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Customer: Alice Actions: -
add_item("Laptop", 1200.00) -
add_item("Mouse", 25.50) - get_total()
```

SAMPLE OUTPUT

```
1225.5
```

Task 2: Rectangle Geometry

Easy • Geometry

PROBLEM STATEMENT

Design a class named `Rectangle` to calculate dimensions and geometric properties of a rectangle.

Your class needs to support:

- Initializing with width and height.
- A method `calculate_area()` to return the area (width × height).
- A method `calculate_perimeter()` to return the perimeter (2 × (width + height)).
- A method `is_square()` to check if the rectangle is a square (returns `True` if width equals height, otherwise `False`).

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Width: 5, Height: 10 Actions: -
calculate_area() - calculate_perimeter() -
is_square()
```

SAMPLE OUTPUT

```
Area: 50 Perimeter: 30 Is Square: False
```

Task 3: Smart Thermostat

Easy • State Logic

PROBLEM STATEMENT

Design a class named `SmartThermostat` to control temperature and power states for a smart home device.

Your class needs to support:

- Initializing with a default target temperature of 22 and a power state of `False` (off).
- Methods `turn_on()` and `turn_off()` to toggle power.
- A method `set_temperature(temp)` which updates target temperature *only if* the thermostat is turned on. If it is off, print an error message.

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Actions: - set_temperature(24) - turn_on() - set_temperature(24)
```

SAMPLE OUTPUT

```
Thermostat is off. Turn it on first. Target temperature set to 24°C.
```

Task 4: Employee Payroll

Easy • Class Methods

PROBLEM STATEMENT

Design a class named `Employee` to track work hours and calculate earnings with potential raises.

Your class needs to support:

- Initializing with employee name and hourly wage (hours worked defaults to 0).
- A method `log_hours(hours)` to add to total hours worked.
- A method `calculate_pay()` returning total pay (hourly wage × hours worked).
- A method `give_raise(amount)` to increase the hourly wage by a specified amount.

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Employee: "John Doe", Wage: 20.0 Actions: - log_hours(40) - calculate_pay() - give_raise(5.0) - calculate_pay()
```

SAMPLE OUTPUT

```
Initial Pay: 800.0 Updated Pay: 1000.0
```

Task 5: Video Game Character

Easy • Attribute Boundaries

PROBLEM STATEMENT

Design a class named `Hero` to manage character health, leveling, and combat status.

Your class needs to support:

- Initializing with a character name, setting health to `100` and level to `1`.
- A method `take_damage(amount)` that subtracts health (ensuring health does not drop below 0).
- A method `heal(amount)` that adds health (ensuring health does not exceed 100).

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Hero Name: "Arthur" Actions: - take_damage(40) - heal(20)
```

SAMPLE OUTPUT

```
Health after damage: 60 Health after healing: 80
```

Task 6: Library Book

Easy • Status Flags

PROBLEM STATEMENT

Design a class named Book to track book availability and checkout status.

Your class needs to support:

- Initializing with a title, author, and an initial checkout status of `False`.
- A method `check_out()` which changes status to `True` if available, or returns a warning if already checked out.
- A method `return_book()` which changes status back to `False`.

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Book: "1984" by "George Orwell" Actions: - check_out() - check_out() - return_book() - check_out()
```

SAMPLE OUTPUT

```
Book successfully checked out. Sorry, this book is already checked out. Book successfully checked out.
```

Task 7: Playlist Manager

Medium • External Modules

PROBLEM STATEMENT

Design a class named `Playlist` to manage a collection of songs in a music playlist.

Your class needs to support:

- Initializing with a playlist name and an empty list of songs.
- A method `add_song(song_title)` to append a song to the playlist.
- A method `play_random()` which uses a random selection approach (e.g. `random.choice`) to pick and return a song from the playlist.

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Playlist Name: "Workout Mix" Actions: - add_song("Eye of the Tiger") - add_song("Stronger") - play_random() [Suppose it selects "Stronger"]
```

SAMPLE OUTPUT

```
Now playing: Stronger
```

Task 8: Car Fuel Mileage

Medium • Validation Logic

PROBLEM STATEMENT

Design a class named `Car` to track fuel consumption and distance traveled.

Your class needs to support:

- Initializing with an initial fuel level (gallons) and efficiency (miles per gallon).
- A method `drive(miles)` which computes required fuel, checks if enough fuel exists, reduces fuel level, and updates state. If fuel is insufficient, prints an error message and does not drive.

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Fuel: 2 gallons, Efficiency: 10 mpg Actions: - drive(15) - drive(10)
```

SAMPLE OUTPUT

```
Remaining fuel: 0.5 gallons Not enough fuel to drive that distance!
```

Task 9: Accumulator Calculator

Easy • Accumulator State

PROBLEM STATEMENT

Design a class named `Accumulator` to maintain a running numeric state through calculations.

Your class needs to support:

- Initializing a `current_value` attribute to 0.
- Methods `add(n)`, `subtract(n)`, and `multiply(n)` to modify the current value.
- A method `clear()` to reset the current value back to 0.

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Actions: - add(10) - multiply(3) - subtract(5)
```

SAMPLE OUTPUT

```
Current Value: 25
```

Task 10: Todo Task Manager

Medium • Collection Filtering

PROBLEM STATEMENT

Design a class named `TodoManager` to keep track of personal to-do tasks and completion statuses.

Your class needs to support:

- Initializing with an owner name and an internal storage structure for tasks.
- A method `add_task(task_name)` to add a new task as incomplete.
- A method `complete_task(task_name)` to mark a specific task as complete.
- A method `show_pending()` to display all tasks that are still incomplete.

INPUT / OUTPUT FORMAT

SAMPLE INPUT

```
Owner: "Bob" Actions: - add_task("Learn Python Classes") - add_task("Build a Portfolio") - complete_task("Learn Python Classes") - show_pending()
```

SAMPLE OUTPUT

```
Pending Tasks: - Build a Portfolio
```